



Article

A Controlled Comparative Evaluation of Infrastructure as Code Tools: Deployment Performance and Maintainability Across Terraform, Pulumi, and AWS CloudFormation

Damir Regvart, Ivan Vlahović and Mislav Balković



Article

A Controlled Comparative Evaluation of Infrastructure as Code Tools: Deployment Performance and Maintainability Across Terraform, Pulumi, and AWS CloudFormation

Damir Regvart ^{*}, Ivan Vlahović and Mislav Balković 

Department of Cybersecurity and System Engineering, Algebra Bernays University, 10000 Zagreb, Croatia; ivlaho2@algebra.hr (I.V.); mislav.balkovic@algebra.hr (M.B.)

^{*} Correspondence: damir.regvart@algebra.hr

Abstract

Infrastructure as Code (IaC) underpins automated cloud provisioning in modern DevOps environments; however, controlled comparative evaluations of leading IaC tools under identical conditions remain limited. This study presents a controlled comparative evaluation of Terraform, Pulumi, and AWS CloudFormation within a standardized Amazon Web Services environment. An identical multi-tier architecture was implemented using each tool, and repeated deployment cycles were conducted to observe differences in provisioning duration, removal time, structural maintainability, and operational characteristics. Descriptive statistical analysis across 30 controlled repetitions indicates that Terraform and Pulumi achieve comparable deployment performance, whereas CloudFormation requires more than twice the average provisioning time under the conditions evaluated. Removal durations were similar across tools but remained longest for CloudFormation. Structural analysis reveals trade-offs between declarative modular design, programmatic flexibility, and native cloud integration. The study provides a controlled, comparative framework to support evidence-based selection of IaC tools in production-oriented cloud environments.

Keywords: Infrastructure as Code; cloud automation; Terraform; Pulumi; AWS CloudFormation; empirical evaluation; analysis; deployment performance

1. Introduction

Contemporary software systems rely on automated, reproducible infrastructure provisioning to maintain continuous delivery, swift scalability, and operational resilience. IaC facilitates automation by specifying infrastructure using version-controlled, machine-readable artifacts that seamlessly interface with DevOps processes. As cloud environments become more complex, the operational consequences of IaC tooling choices become increasingly critical. In distributed cloud systems, orchestration latency and provisioning determinism directly influence pipeline throughput, recovery time objectives, and infrastructure elasticity.

Terraform, Pulumi, and AWS CloudFormation are three prevalent IaC solutions with distinct design principles. Terraform uses a declarative domain-specific language; Pulumi uses general-purpose programming languages to define infrastructure; and CloudFormation offers a native managed service closely integrated with the AWS ecosystem. While each tool may offer functionally comparable designs, their operational attributes vary in terms of deployment delay, failure management, and maintainability.



Academic Editor: Alexander Barkalov

Received: 21 February 2026

Revised: 12 March 2026

Accepted: 19 March 2026

Published: 19 March 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Notwithstanding their widespread industrial use, comprehensive quantitative evaluations of these instruments remain limited. Many current evaluations rely on qualitative analysis, limited experimentation, or anecdotal evidence. Few studies employ replication across multiple operational dimensions. As a result, current comparisons are often not generalizable and offer less decision-making support for practitioners. The selection of IaC tools significantly affects deployment efficiency and long-term maintainability; thus, the lack of controlled, multifaceted empirical evaluation limits the depth of evidence available to support performance-sensitive decision-making in applied cloud engineering research.

This study addresses the gap by conducting a controlled comparative evaluation of Terraform, Pulumi, and AWS CloudFormation under defined conditions within AWS. An identical multi-tier web application architecture was implemented using each tool, and each configuration was deployed and dismantled 30 times. Descriptive statistical analysis across the repeated cycles quantified distributional characteristics, including variability and dispersion patterns. Deployment performance, stability indicators, and maintainability characteristics were observed and compared under standardized conditions.

This study conducts a controlled comparative assessment of three prevalent Infrastructure as Code tools: Terraform, Pulumi, and AWS CloudFormation, to mitigate these constraints. The assessment emphasises operational deployment efficacy, infrastructure removal tendencies, and structural maintainability attributes under uniform architectural and climatic settings. The study seeks to provide replicable empirical data to inform the selection of Infrastructure as Code tools in production-oriented cloud settings by conducting multiple deployment cycles of a uniform multi-tier cloud architecture. The subsequent inquiries direct the research:

- RQ1: How do the tools differ in observed operational efficiency in regulated cloud deployments?
- RQ2: Does the IaC paradigm meaningfully influence maintainability attributes and deployment stability?

Operational efficiency primarily concerns deployment duration and removal performance under controlled conditions, whereas maintainability and stability pertain to the code's structural characteristics and failure behavior during deployment. The contributions of this study are:

1. A comparative evaluation of three widely used IaC tools under identical architectural and environmental conditions.
2. Observed differences in deployment and removal performance across tools.
3. A structured analysis of maintainability and operational characteristics across declarative, programmatic, and native cloud paradigms.
4. A transparent methodological framework supporting reproducible comparative studies in cloud infrastructure engineering.

The remainder of this paper is structured as follows. Section 2 reviews related work on Infrastructure as Code and empirical evaluations of cloud automation tools. Section 3 presents the research methodology. Section 4 reports the results of comparative analysis. Section 5 discusses the implications of the findings and outlines threats to validity. Finally, Section 6 concludes the paper and suggests directions for future research.

2. Related Works

Research on Infrastructure as Code has increased significantly over the last five years, with a focus on defect characterization, security analysis, test automation, governance standards, and tool benchmarking. Nonetheless, methodologically rigorous, multidimensional empirical evaluations of prominent IaC methods remain few.

Numerous studies have investigated problem patterns and quality characteristics of Infrastructure as Code scripts. Rahman et al. categorized systematic defects in configuration artifacts, revealing that IaC scripts exhibit fault distributions that differ from those of conventional application code [1]. Subsequent research expanded this technique to defect prediction, demonstrating that structural and process measurements can pinpoint error-prone components of infrastructure [2]. Extensive empirical studies have confirmed associations between product metrics and defect susceptibility in Infrastructure-as-Code repositories [3]. Although these contributions enhance understanding of script-level quality, they primarily assess static attributes rather than runtime performance under controlled deployment conditions.

Security research has identified structural vulnerabilities in infrastructure definitions. Fischer et al. introduced static analysis methods to identify vulnerabilities in Infrastructure-as-Code setups, revealing prevalent misconfigurations in real-world repositories [4]. Expanded vulnerability taxonomies enhanced previous classifications and recorded persistent misconfiguration patterns within Infrastructure as Code ecosystems [5]. While these studies enhance security assurance methods, they do not evaluate the efficacy of comparable tools and therefore cannot guide performance-sensitive tool selection decisions.

The testing and validation of IaC programs constitute a significant area of ongoing research. Sokolowski et al. presented automated testing methodologies specifically designed for IaC settings and supplied curated datasets that facilitate reproducible large-scale analyses [6,7]. Comprehensive studies of Infrastructure as Code lifecycle management have highlighted governance frameworks, modularization techniques, and quality assurance in DevOps pipelines [8]. These initiatives improve reliability engineering but fail to thoroughly evaluate the operational attributes of rival IaC models.

The investigation of operational efficiency in cloud automation has also been conducted. Sharma et al. showed that infrastructure provisioning latency directly affects CI/CD throughput and overall system responsiveness [9]. Investigations into configuration quality measurements have further delineated quantifiable correlations between code structure and maintainability results [10]. Enterprise-focused frameworks for multi-cloud IaC adoption prioritize modular architecture and policy-as-code implementation to enhance governance and operational uniformity [11]. Nevertheless, these studies generally lack controlled experimental replication when assessing particular IaC approaches.

Comparative assessments of Infrastructure as Code tools are limited and methodologically restricted. Vaggu delineated the trade-offs between Terraform and Pulumi in terms of extensibility and provisioning behavior [12], whereas other analyses compared Terraform with CloudFormation from architectural and ecosystem perspectives [13]. Numerous comparative studies rely on limited deployment cycles, descriptive analyses, or single-metric evaluations. Formal statistical modeling and effect size reporting remain relatively uncommon in comparative IaC evaluations.

Recent studies have investigated AI-enhanced IaC generation and infrastructure reconciliation. The Multi-IaC-Eval benchmark assessed automated template synthesis across many formats, uncovering semantic and validation issues in model-driven infrastructure production [14]. AI-driven reconciliation agents have examined the automated identification and rectification of configuration drift [15]. Although these advancements pertain to automation processes, they do not eliminate the necessity for controlled, methodically rigorous comparisons of existing production tools.

A recent study has investigated the growing operational and lifecycle challenges associated with deploying Infrastructure as Code. Empirical studies on IaC dependency management reveal that infrastructure configurations often experience technical delays due to postponed module upgrades, which can last several months in ongoing projects.

This delay might introduce compatibility risks and unresolved vulnerabilities in infrastructure deployments, underscoring the need for enhanced tooling support and lifecycle management systems for IaC modules [16].

Simultaneously, research has investigated automated methods to enhance the dependability and security of Infrastructure as Code settings. Automated threat-modeling frameworks like TerrARA analyze Terraform configuration files to generate infrastructure models and systematically detect potential security issues in cloud deployments. Incorporating these methodologies into CI/CD pipelines facilitates ongoing security evaluation of infrastructure configurations and mitigates the likelihood of misconfiguration during automated provisioning [17].

Recent studies have focused on the automated remediation and quality assurance of Infrastructure as Code (IaC) programs. Frameworks like InfraFix demonstrate that automated program-repair methodologies can effectively detect and rectify configuration errors in Infrastructure as Code (IaC) scripts, achieving high repair success rates across extensive experimental datasets. These methodologies exemplify the increasing significance of automated analysis and correction systems in enhancing infrastructure resilience [18].

Recent research on infrastructure automation in cloud-native data platforms underscores that modular architectures, enhanced state management systems, and cohesive dependency management can markedly enhance operational reliability and scalability in contemporary cloud environments [19].

Notwithstanding these advancements in Infrastructure as Code security analysis, lifecycle management, and automated configuration remediation, systematic empirical comparisons of prevalent Infrastructure as Code technologies under standardized deployment conditions are still few. Therefore, additional controlled experiments are necessary to elucidate the operational distinctions across leading IaC frameworks. This study examines the disparity by conducting a controlled comparative analysis of Terraform, Pulumi, and AWS CloudFormation in a consistent experimental setting.

The literature indicates significant advancements in Infrastructure as Code quality analysis, security evaluation, and automation methodologies. Nonetheless, experimentation with adequate replication and comprehensive, multi-factor assessment across operational dimensions remains rare. Rigorous, controlled empirical evaluations of deployment performance, stability, and maintainability across prominent IaC paradigms remain limited in the literature. This research enhances the domain by integrating repeated controlled deployments with structured comparative analysis to provide reproducible, decision-quality evidence for selecting IaC tools.

3. Materials and Methods

This section outlines the experimental framework used to assess operational variances across IaC tools. The study aimed to isolate tool-specific effects under controlled conditions while ensuring methodological rigor, measurement precision, and reproducibility. All architectural, environmental, and procedural variables were standardized across tools. The methodology combines controlled cloud experimentation with structured comparative analysis to facilitate comparative evaluation of practical operational differences.

3.1. Experimental Design

A controlled repeated-deployment experimental design was employed. The primary comparison factor was the selected IaC tool (Terraform 1.12.2, Pulumi 3.186.0, AWS CloudFormation). Measured characteristics included deployment performance, stability, and maintainability. Terraform, Pulumi, and AWS CloudFormation were chosen for their prominence as Infrastructure as Code tools in modern cloud engineering, each offering

a distinct approach to infrastructure automation. Terraform is extensively utilised as a declarative, domain-specific Infrastructure as Code solution for multi-cloud environments. Pulumi embodies a programmatic Infrastructure as Code (IaC) methodology that utilises general-purpose programming languages for defining infrastructure. AWS CloudFormation offers a provider-native orchestration approach that is closely linked with the AWS environment. Assessing these three technologies facilitates a comparison of declarative configuration models, programmatic infrastructure definitions, and provider-managed orchestration frameworks.

Each tool provides an identical multi-tier cloud architecture within Amazon Web Services (AWS). Every configuration was deployed and destroyed 30 times, yielding 90 total experimental runs. Each cycle began from a fully cleared cloud state to minimize cross-run interference between observations.

All experiments were conducted:

- Within the same AWS account;
- In the same region (eu-central-1);
- Using identical credentials and permission levels;
- From a dedicated Fedora Linux virtual machine;
- Over a stable wired network connection.

No concurrent provisioning tasks were executed during measurement periods. This design isolated tool-specific behavior while minimizing environmental variance.

3.2. Infrastructure Specification

The assessed architecture emulated a production-level web application environment comprising:

- Virtual Private Cloud (VPC);
- Public and private subnets spanning two Availability Zones;
- Internet Gateway and dual NAT Gateways;
- Application Load Balancer;
- Auto Scaling Group of EC2 Instances;
- Amazon RDS MySQL Database.

Figure 1 shows the exact visual representation of the infrastructure:

All resource characteristics, including instance kinds, storage sizes, scaling thresholds, network topologies, and security protocols, were rigorously standardized among implementations. The definitions of infrastructure varied solely in the syntactic representation each tool mandated.

The architecture included interdependent components (e.g., NAT readiness before instance initialization) to replicate authentic operational complexity. This circumvented simplistic deployment scenarios and guaranteed significant tool stress under production-like settings.

Remote state backends tailored to each tool were employed to embody collaborative operational practices and ensure consistent state tracking.

3.3. Operational Variables and Measurement Definitions

All dependent variables were clearly defined to guarantee construct clarity and reproducibility.

3.3.1. Evaluating Deployment Performance

The subsequent temporal metrics were documented:

- Initialization time refers to the delay from the initiation of a command to the successful initialization of the environment.

- Planning time—the duration necessary to compute the execution plan.
- Provisioning time refers to the interval between execution confirmation and the complete readiness of the infrastructure.
- Destruction time—the period necessary for total resource dismantlement.
- Total deployment duration—the elapsed time from the initiation of the provisioning command to confirmation of application availability.

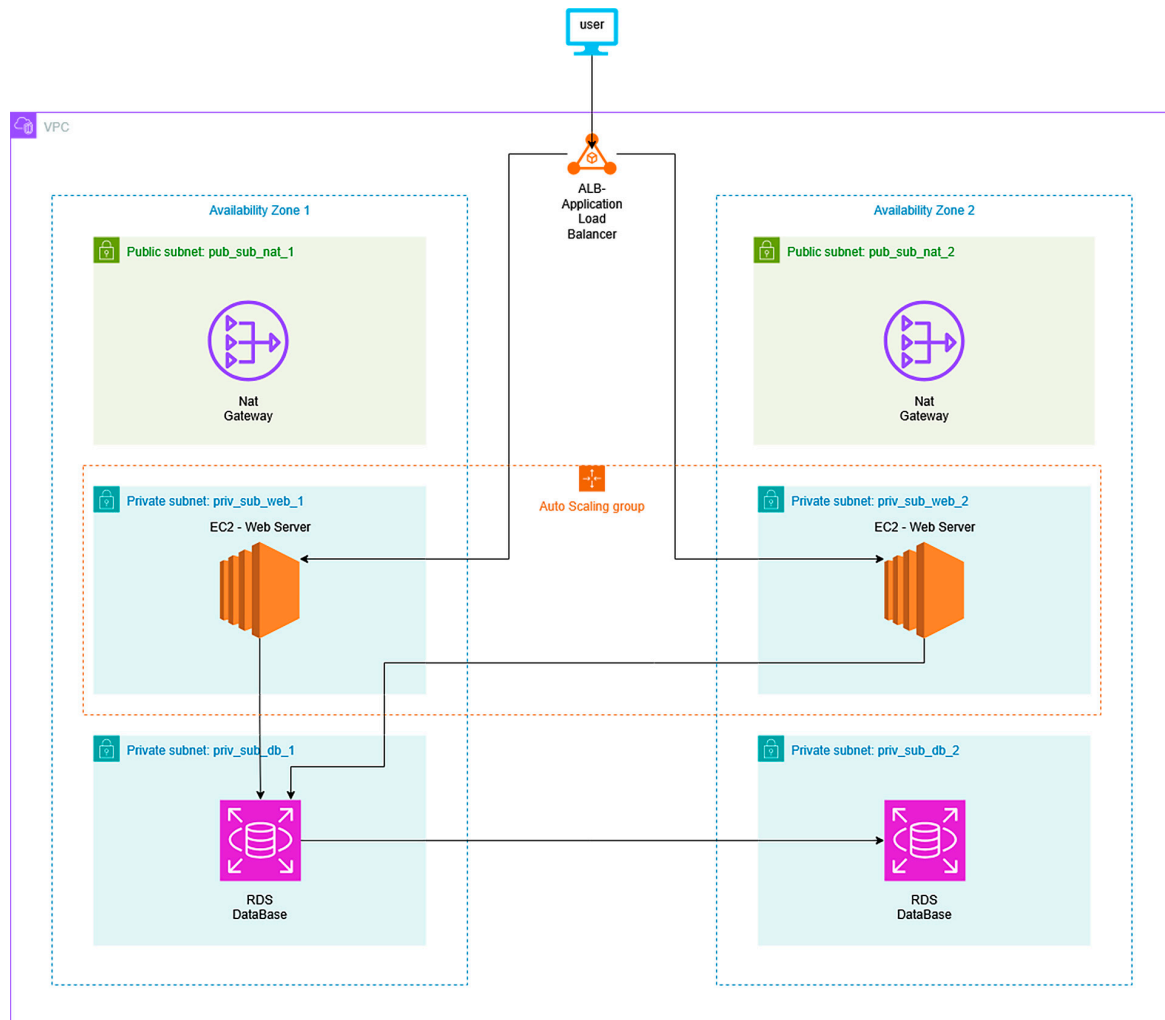


Figure 1. Architecture of the evaluated cloud infrastructure, including Virtual Private Cloud (VPC), Application Load Balancer (ALB), Auto Scaling EC2 instances, Amazon RDS database, NAT gateways, and public and private subnets across two availability zones.

Timestamp resolution was recorded at second-level precision using automated shell logging to ensure consistent measurement capture across runs.

To ensure measurement consistency, all temporal measurements were collected using automated shell scripts that recorded Unix timestamps immediately before and after each operational phase of the deployment process. Each phase was executed via the command-line interfaces of the corresponding tools (Terraform CLI, Pulumi CLI, and AWS CLI for CloudFormation). At the same time, the logging script recorded the start and end times for the startup, planning, provisioning, and destruction stages.

The timestamps were recorded in structured log files and subsequently analyzed to calculate the duration of each experimental run, in seconds. All measurements were acquired in the same execution environment to eliminate clock drift or synchronization

discrepancies. This automated method guaranteed uniform data collection across all deployment iterations while simultaneously reducing observer bias.

Deployment and destruction durations were recorded via automated shell logging scripts that captured timestamps at the start and end of each operational phase. The measurements depended on synchronised system time from the execution environment to guarantee temporal consistency between executions. All timing data were automatically collected and recorded in structured log files, thereby reducing observer bias and ensuring consistent measurement over multiple deployment cycles.

3.3.2. Stability

Stability was assessed utilizing:

- Binary failure event during provisioning;
- Existence of partial deployment states;
- Errors in dependency resolution.

Failed attempts were documented independently. Performance indicators were derived solely from successful installations, whereas failure frequency informed the stability analysis.

3.3.3. Maintainability

Maintainability was evaluated through a structured comparison of configuration organization, modular decomposition, and code verbosity across tools.

The following aspects were considered:

- Degree of modular separation of infrastructure components;
- Reusability structure (presence of reusable modules or classes);
- Configuration verbosity and structural clarity.

Terraform implementations employed modular HCL definitions with centralized variable management, enabling clear separation of infrastructure components. Pulumi structured infrastructure using Python 3.12.11 classes representing logical units, allowing object-oriented abstraction and parameterization. CloudFormation relied on YAML templates and nested stacks, resulting in longer configuration files and more extensive parameter definitions.

The analysis focused on structural characteristics observable from the implemented configurations rather than formal static code metric modeling.

3.4. Experimental Protocol

Every deployment cycle adhered to a uniform protocol:

1. Validation of an unoccupied AWS environment;
2. Initialization of the tool;
3. Execution of the deployment command;
4. Oversight until verified operational readiness;
5. Documentation of temporal metrics;
6. Complete infrastructure devastation;
7. Validation of comprehensive cleanup.

All cycles were performed in succession. Manual intervention was only allowed in the event of a failure. Residual state artifacts were confirmed to be absent between cycles.

To mitigate bias:

- Identical hardware and software systems were employed;
- Network conditions remained stable;
- AWS service limits were observed to avert throttling;

Deployment scripts remained unaltered throughout the experimentation.

3.5. Analytical Approach

The gathered measurements were examined using systematic descriptive comparison. For each tool, the average deployment, removal, and other operational durations were calculated across several deployment cycles. The investigation concentrates on performance trends seen under controlled conditions. This strategy facilitates the identification of recurrent operational disparities while ensuring methodological transparency. Descriptive statistics were computed from the collected measurements, including mean values, standard deviation, range, skewness, and kurtosis, to characterize the distribution of observed deployment durations.

All tools were assessed under uniform architectural, environmental, and procedural conditions. Thus, the observed discrepancies are due to tool performance rather than variations in infrastructure.

3.6. Validity Considerations

Environmental control, multiple independent deployments, uniform architectural specifications, and standardized credentials enhanced internal consistency.

Operational consistency was established through precise operational definitions and automated data collection.

The production-representative architecture pattern supports operational consistency. While the experiments were AWS-based, the architectural model reflects common cloud deployment structures, thereby supporting broader generalizability in similar contexts.

3.7. Methodological Summary

This study employs a controlled comparative experimental design to evaluate operational differences among widely used IaC tools. By standardizing infrastructure architecture, execution environment, deployment procedures, and measurement criteria, the methodology isolates tool-specific behavior under consistent conditions. Repeated deployment cycles enable robust observation of performance, stability, and structural characteristics.

This framework provides a transparent and reproducible basis for examining practical differences across IaC paradigms while maintaining methodological clarity.

4. Results

This section presents the empirical findings of the controlled comparative evaluation, focusing on deployment performance, infrastructure removal duration, and maintainability characteristics of the evaluated IaC tools. Results are organized according to deployment performance, infrastructure removal duration, structural maintainability, and qualitative operational characteristics.

4.1. Deployment Performance

Deployment performance was evaluated by measuring the time required to provision and subsequently remove the complete multi-tier infrastructure under identical environmental conditions. The following subsections present the observed provisioning and removal durations for each IaC tool, allowing direct comparison of operational efficiency.

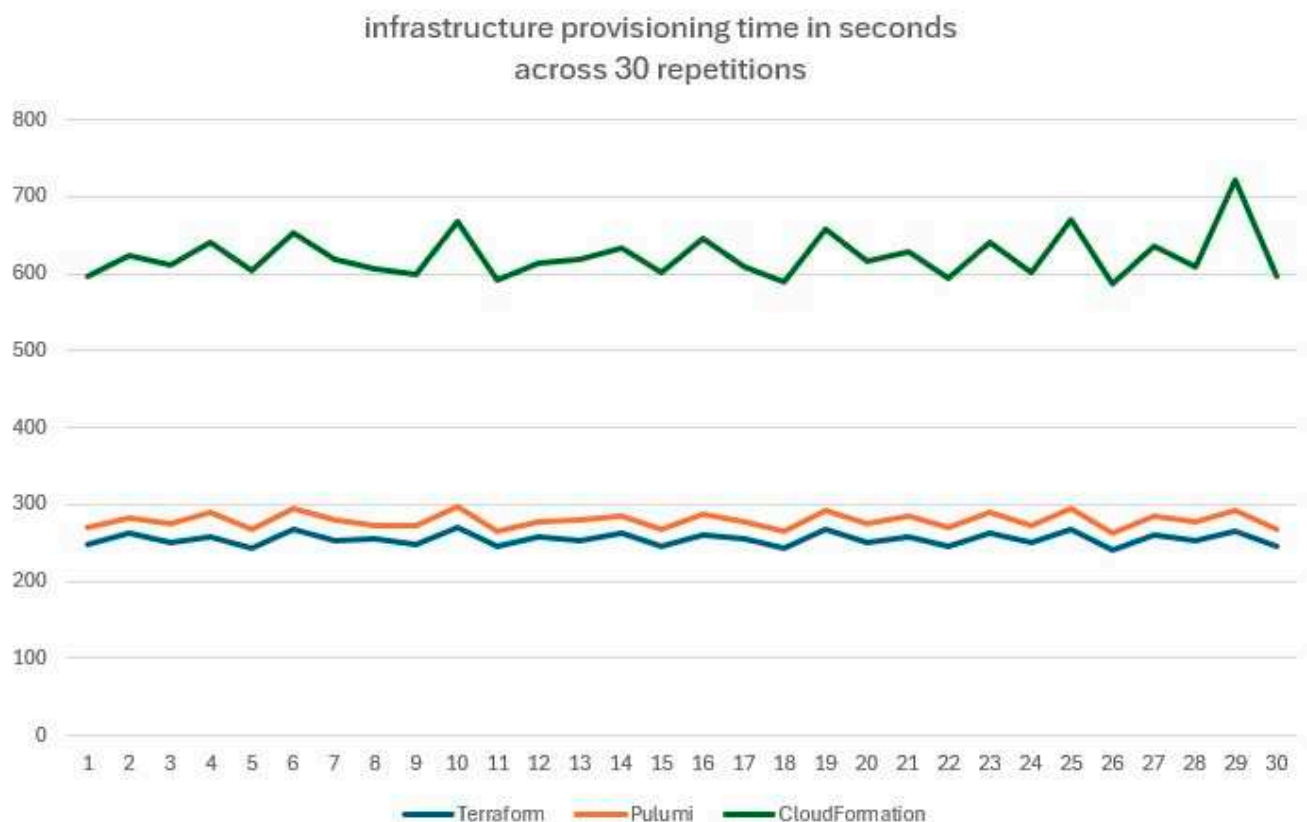
4.1.1. Infrastructure Provisioning Time

The average time required to provide the complete multi-tier infrastructure is presented in Table 1:

Table 1. Average infrastructure provisioning time.

Tool	Average Provisioning Time
Terraform	4 min 15 s (253 s)
Pulumi	4 min 37 s (279 s)
CloudFormation	10 min 18 s (623 s)

The set of 30 measurements performed showed various levels of deviation, as visible in Figure 2:

**Figure 2.** Deployment time across 30 repetitions.

Terraform exhibited a mean provisioning time of 253.33 s (SD = 8.42), while Pulumi averaged 279.37 s (SD = 9.89). CloudFormation showed substantially longer provisioning times with greater variability (mean = 622.97 s, SD = 29.92). Distributional analysis indicates stable execution for Terraform and Pulumi (skewness < 0.2), whereas CloudFormation displayed higher skewness and kurtosis, suggesting occasional longer provisioning events. The relatively low standard deviations for Terraform and Pulumi indicate stable deployment behavior across repetitions, whereas CloudFormation showed greater dispersion. Expressed as relative performance:

- CloudFormation required approximately 2.4× the provisioning time of Terraform;
- CloudFormation required approximately 2.2× the provisioning time of Pulumi.

These differences were observed across repeated deployment cycles. Statistical analysis for the deployment phase is shown in Table 2. The next subsection presents the analysis of the removal phase.

Table 2. Statistical analysis of the infrastructure deployment phase.

Tool	Mean (s)	Median	SD	Variance	Min	Max	Range	Skewness	Kurtosis
Terraform	253.33	254.5	8.42	70.85	242	271	29	0.20	−1.05
Pulumi	279.37	278.5	9.89	97.90	264	297	33	0.17	−1.15
CloudFormation	622.97	615.0	29.92	895.41	587	722	135	1.44	2.75

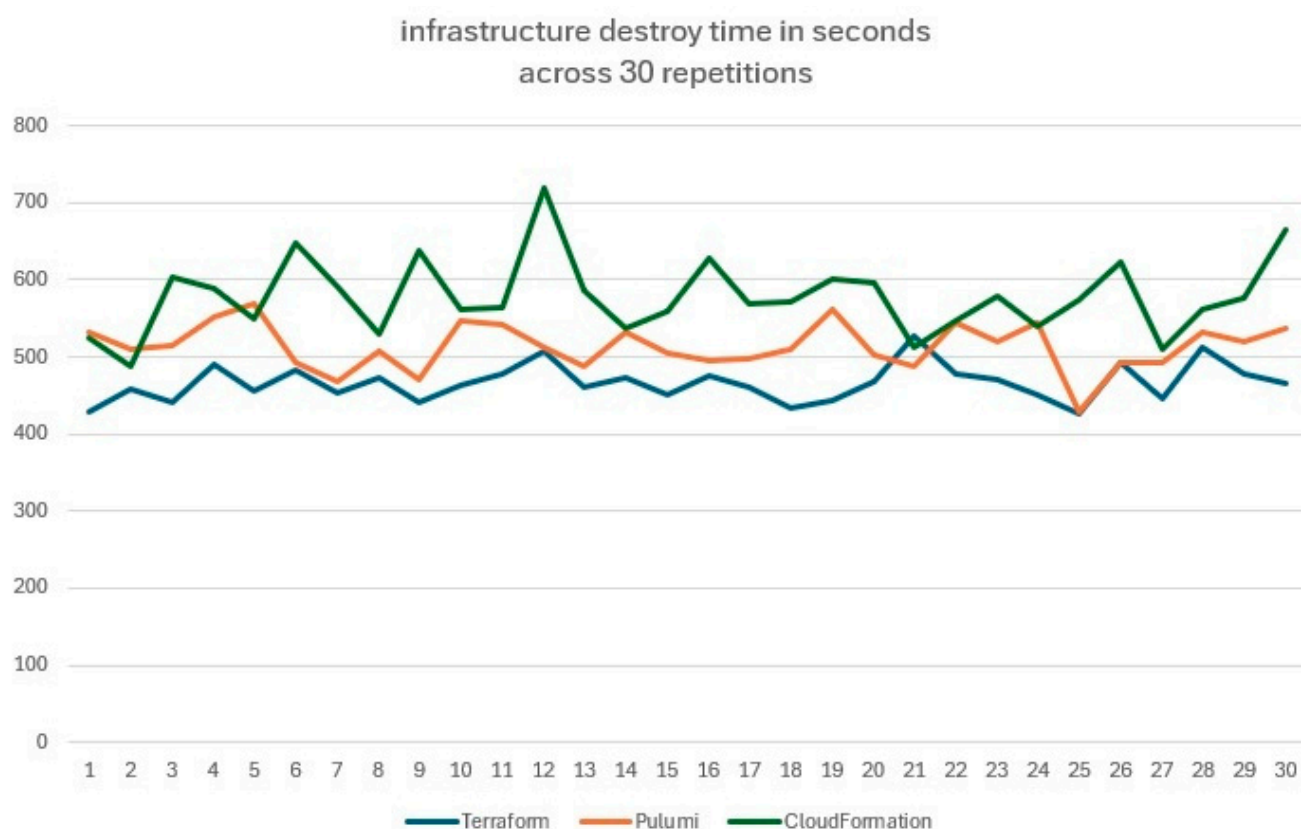
4.1.2. Infrastructure Removal Time

All tools required slightly more time to remove infrastructure than to provision it, as described in Table 3:

Table 3. Average infrastructure removal time.

Tool	Average Removal Time
Terraform	7 min 42 s (462 s)
Pulumi	8 min 37 s (517 s)
CloudFormation	9 min 35 s (575 s)

Infrastructure removal exhibited greater variability across repetitions, as shown in Figure 3:

**Figure 3.** Destroy time across 30 repetitions.

Terraform exhibited a mean destruction time of 466.23 s (SD = 23.98), Pulumi 513.73 s (SD = 30.68), and CloudFormation 578.40 s (SD = 49.52), indicating progressively increasing variability across tools. Removal-phase distributions exhibited lower skewness than provisioning, indicating fewer extreme outlier events during teardown operations. Statistical analysis of the destruction phase is available in Appendix A, Table A1.

Removal durations were more similar across tools than provisioning times, although CloudFormation remained the slowest overall.

4.2. Code Structure and Maintainability

Maintainability was assessed through a systematic analysis of configuration modularity, abstraction techniques, dependency management, and state-handling attributes identified throughout the implementation. The analysis emphasizes configuration-level attributes rather than automated static code measures, consistent with the study's implementation-based comparison framework.

4.2.1. Structural Modularity

All tools facilitated the decomposition of the infrastructure into logical components for networking, security, computation, load balancing, and databases. The strategies employed to attain modular separation varied.

Terraform utilized six distinct HCL modules (vpc, nat, alb, sec_group, web, database), each contained in its own directory and managed by a single orchestration file. This framework facilitated a clear delineation of responsibilities and localized adjustments without disrupting inter-component operations.

Pulumi achieved a similar logical separation by using Python ComponentResource classes. Every infrastructure layer was encased behind a reusable class abstraction, facilitating object-oriented modularity and customization. This method combines infrastructure design with recognized principles of software engineering structure.

CloudFormation achieved modularization by using YAML stacks stacked and interconnected via a master template. Although logically organized, cross-stack parameter transmission and template verbosity increased structural complexity compared with module- and class-based implementations.

4.2.2. Abstraction and Reusability

Terraform prioritizes declarative precision via HCL modules and variable parameterization. The restricted imperative constructs reduce logical complexity while maintaining a consistent infrastructure specification.

Pulumi leverages the full features of a programming language (Python in this case), including loops, functions, classes, and external integrations. This offers significant flexibility in abstraction and integration with software development workflows, though it may entail a cognitive burden in general-purpose programming environments.

CloudFormation depends on declarative YAML templates. Reuse is achieved through nested stacks; however, the flexibility of abstraction is constrained without external frameworks such as the AWS CDK. Template verbosity escalates with architectural complexity, potentially impacting long-term readability.

4.2.3. Management of State and Operational Resilience

The attributes of state management directly affect repeatability and operational sustainability.

Terraform used a remote S3 backend with state locking, ensuring transparency and collaborative security, but requiring meticulous setup discipline.

Pulumi used centralized cloud-state storage with versioning. Dependency resolution was predominantly automated, while some infrastructural dependencies necessitated explicit enforcement.

CloudFormation internally handles state via AWS stack techniques. An important operational feature was automatic rollback in the event of provisioning failure, minimizing manual remediation efforts and improving recovery resilience.

These structural distinctions reflect differences in abstraction layering and orchestration responsibility between tool-internal execution engines and user-defined configuration logic.

The comparative overview in Table 4 highlights that differences in maintainability arise primarily from architectural philosophy rather than from implementation correctness. While all tools support modular decomposition and reproducible state management, they diverge in abstraction flexibility, dependency transparency, and recovery mechanisms. These distinctions shape how infrastructure complexity scales over time and influence the long-term sustainability of configuration under evolving production requirements.

Table 4. Structured maintainability comparison of evaluated IaC tools.

Dimension	Terraform	Pulumi	AWS CloudFormation
Modularity mechanism	Separate HCL modules per infrastructure layer	ComponentResource class-based modularity	Nested YAML stacks
Separation of concerns	Directory-based module isolation	Encapsulated class abstractions	Logical segmentation via stack templates
Abstraction level	Declarative with limited imperative constructs	Full programming language expressiveness	Declarative template-based
Reusability model	Module reuse via variables and outputs	Class reuse and programmable abstractions	Nested stack reuse
Dependency handling	Explicit when required (depends_on)	Primarily automatic with optional enforcement	Stack-based automatic resolution
State management	External state (S3 backend with locking)	Centralized service-based state	AWS-managed stack state
Rollback capability	Manual recovery required	Manual recovery required	Built-in automatic rollback
Configuration verbosity	Moderate	Moderate (implementation-dependent)	High in complex templates
Maintainability orientation	Structured declarative modularity	High abstraction flexibility	Robust native integration with higher verbosity

4.2.4. Maintainability Synthesis

The maintainability trade-offs among the assessed tools were predominantly influenced by abstraction philosophy and state management architecture, rather than solely by deployment efficiency. Terraform emphasizes declarative, modular organization and explicit state management. Pulumi highlighted the flexibility of abstraction and its connection with programming paradigms. CloudFormation offered robust AWS-native state management and rollback capabilities, although it necessitated more elaborate setup frameworks.

In controlled experimental conditions, all three tools exhibited stable configuration behavior and functional correctness; however, their structural characteristics indicate varying long-term maintainability profiles based on organizational preferences for declarative clarity, programmatic flexibility, or managed-service robustness.

4.3. Operational Usability Observations

Qualitative operational differences were observed during deployment execution. For example, Terraform and Pulumi use concise command-line interfaces to plan, apply, and destroy infrastructure. In contrast, CloudFormation requires longer command syntax with multiple parameters for stack management and output inspection.

The differences in command syntax can be illustrated with simple examples of common deployment commands used by each tool:

1. Terraform:
 - terraform init
 - terraform plan
 - terraform apply
2. Pulumi:
 - pulumi preview
 - pulumi up
3. CloudFormation:
 - aws cloudformation deploy--template-file template.yaml--stack-name demo-stack

As shown in the examples above, CloudFormation commands typically require additional parameters for stack management, increasing command verbosity compared with Terraform and Pulumi. Monitoring output readability also differed:

- Terraform and Pulumi provided concise, readable logs with clear progress indication.
- CloudFormation output required a more detailed log inspection to identify stack events.

These differences did not affect functional correctness but influenced operational ergonomics.

4.4. Summary of Observed Differences

Under identical infrastructure and environmental conditions, the following key observations were made:

- Terraform and Pulumi demonstrated comparable provisioning performance;
- CloudFormation required more than double the provisioning time;
- Removal durations were similar across tools but remained longest for CloudFormation;
- Terraform exhibited more concise modular definitions under the implemented architecture, less so in CloudFormation;
- Pulumi offered the highest programmatic flexibility.

All three tools successfully provisioned the target infrastructure without persistent configuration inconsistencies.

5. Discussion

This section analyzes the ramifications and constraints of the controlled comparative evaluation outlined in this work. The findings offer a structured overview of the operational distinctions among Terraform, Pulumi, and AWS CloudFormation; however, methodological and contextual limitations constrain the scope and generalizability of the results.

5.1. Scope of Experimentation and Architectural Representativeness

The assessment was performed utilizing a singular production-representative multi-tier web architecture. The architecture includes realistic infrastructure components such as load balancing, autoscaling, database services, and multi-zone networking; however, it does not cover the full range of potential cloud deployment scenarios.

Infrastructure as Code technologies may have varying operating characteristics across distinct architectural patterns, including:

- Architectures without servers;
- Container orchestration systems;
- Event-driven architectures;
- Extensive microservices ecosystems;
- Deployments across multiple regions;

- Highly stateful distributed architectures.

Thus, the noted performance and structural variations must be understood in relation to the specific architectural design under assessment. The results do not indicate the universal superiority of any tool across all infrastructure types.

5.2. Specificity of Cloud Providers

All trials were performed within Amazon Web Services (AWS). Terraform and Pulumi are multi-cloud platforms, but CloudFormation is specific to AWS. The native integration of CloudFormation may affect deployment behavior in varying ways across distinct resource types and service APIs. No persistent deployment failures were observed across the 30 repeated cycles for any tool.

Performance characteristics, error handling, and provisioning latency may differ among cloud providers due to variations in API responsiveness, internal orchestration methods, and resource provisioning frameworks. Consequently, the results should not be readily generalized to Azure, Google Cloud Platform, or hybrid cloud environments without further controlled assessment.

5.3. Comparative Methodology

This study employs a comparative methodology instead of formal inferential statistical modeling. Despite conducting several deployment cycles to improve observational robustness, the research objective focused on identifying consistent operational trends and relative performance disparities under controlled conditions, rather than engaging in probabilistic generalization.

The analytical focus is on reliable, controlled comparisons and the practical significance of detected discrepancies. This methodological approach corresponds with the engineering-focused aim of assessing operational performance under uniform infrastructure settings. Inferential modeling could further refine statistical interpretation; however, the study prioritizes controlled engineering comparison.

From an operational standpoint, deployment performance may also affect the financial expenditure of automated infrastructure pipelines. Numerous CI/CD environments employ execution runners that are charged on a per-minute basis. Under the specified experimental settings, CloudFormation provisioning necessitated approximately 2.4 times the execution duration of Terraform. In extensive automated deployment settings, such discrepancies may lead to higher operational expenses for infrastructure orchestration pipelines.

5.4. Environmental and Temporal Limitations

All deployments have been executed:

- In a singular AWS region, utilizing a separate virtual computer;
- In steady network conditions;
- Absent concurrent provisioning load.

Cloud service latency and API performance may fluctuate over time due to provider load, geographical demand variations, and service throttling. The repeated-deployment methodology mitigates transient abnormalities; nonetheless, the study fails to account for long-term temporal variability or for performance variation across multiple regions.

Moreover, the functionality of AWSs continues to develop. Enhancements to orchestration pipelines or backend optimizations may affect comparable outcomes in subsequent service versions.

5.5. Limitations of Maintainability Assessment

Maintainability was assessed using a systematic comparison of configuration organization, modularization patterns, and code verbosity. While visible structural aspects were examined, formal static code metrics (such as cyclomatic complexity, dependency graphs, and maintainability indices) were not calculated.

Furthermore, maintainability encompasses socio-technical aspects that are not addressed in this assessment, including:

- Team cohesion;
- Ecosystem sophistication;
- Support from the community;
- Quality of documentation;
- Integration of toolchain;
- Learning curve.

Consequently, the structural comparison illustrates configuration-level attributes rather than overall organizational sustainability.

5.6. Security and Compliance Considerations in Infrastructure as Code

In addition to operational performance and maintainability, security and privacy constraints are becoming increasingly significant in cloud installations driven by Infrastructure as Code. Infrastructure as Code (IaC) technologies allow businesses to formalize infrastructure configurations, enhancing consistency and minimizing human error; however, inadequately defined infrastructure may create security vulnerabilities or compromise critical resources. Consequently, contemporary Infrastructure as Code ecosystems integrate multiple approaches to provide secure infrastructure provisioning and enforce compliance.

Terraform and Pulumi facilitate policy-as-code frameworks that enable enterprises to implement security policies across the infrastructure deployment process. Tools like Open Policy Agent (OPA) and Sentinel facilitate the automatic validation of infrastructure definitions against organizational security policies before deployment. These methods can prevent the establishment of insecure configurations, such as publicly accessible storage services, overly permissive network policies, or missing encryption settings.

AWS CloudFormation offers security measures through its comprehensive integration with core AWSs, including Identity and Access Management (IAM), encryption services, and stack-level access policies. These capabilities enable administrators to implement access controls, apply encryption configurations to managed resources, and regulate permission to modify deployed infrastructure stacks. Moreover, CloudFormation incorporates rollback capabilities that automatically restore infrastructure configurations in the event of deployment mistakes, therefore reducing the risks associated with misconfiguration.

From a secure development standpoint, IaC technologies provide version-controlled infrastructure definitions, hence enabling security audits, peer reviews, and traceability of configuration modifications. This technique enhances transparency and promotes adherence to regulatory frameworks necessitating documented infrastructure management methods. Moreover, integrating IaC workflows with CI/CD pipelines enables automated security screening of configuration files with specialized static analysis tools that identify unsafe infrastructure definitions.

The current study largely emphasizes operational performance and maintainability attributes; however, security and privacy capabilities constitute a significant aspect of Infrastructure as Code adoption. Future experimental research may expand the existing paradigm to assess the efficacy of various Infrastructure as Code (IaC) solutions in facilitating secure configuration validation, automated compliance verification, and safeguarding sensitive infrastructure data across large-scale cloud systems.

5.7. Usability and Qualitative Observations

Operational usability discrepancies were noted throughout deployment execution; however, these observations were qualitative and not based on controlled human-subject experimentation. No user studies, cognitive load assessments, or task-completion experiments were conducted.

Consequently, ergonomic conclusions ought to be regarded as practitioner-informed insights rather than empirically substantiated usability results.

5.8. Risks to External Validity

The architecture, while depicting a realistic industrial environment, is constrained by external validity due to:

Single-cloud-provider environment;

- Unified infrastructure topology;
- Unified execution environment;
- Restricted versions of tools assessed.

Collectively, these constraints limit the interpretative range of the findings to the assessed architectural pattern, execution environment, and cloud provider context. The study does not assert universal performance hierarchies applicable to all infrastructure types, cloud ecosystems, or organizational contexts. Rather, it provides a regulated, replicable comparative evaluation under well-established operational parameters. The study emphasizes methodological clarity by specifically limiting architectural diversity, environmental variability, analytical scope, and maintainability dimensions. The results should be regarded as context-specific operational observations rather than universal performance guarantees, while nevertheless offering significant comparative insights for analogous production-oriented deployment scenarios.

6. Conclusions and Future Work

This study conducted a controlled comparative assessment of three prevalent Infrastructure as Code tools—Terraform, Pulumi, and AWS CloudFormation—under uniform architectural and environmental settings within Amazon Web Services. The research analyzed deployment performance, structural maintainability, and operational characteristics by establishing a production-representative multi-tier infrastructure and conducting iterative deployment cycles.

The results indicate that Terraform and Pulumi exhibit similar provisioning efficiency, whereas CloudFormation requires much longer deployment times under the assessed settings. Removal durations were more consistent among tools; CloudFormation exhibited the longest durations. A structural study identified several paradigms: Terraform focuses on modular, declarative composition; Pulumi provides programmatic abstraction and flexibility; and CloudFormation promotes extensive native integration, leading to greater verbosity. The results demonstrate that selecting Infrastructure as Code (IaC) tools can affect operational deployment dynamics and configuration architecture in production environments. The findings indicate that IaC tool architecture influences orchestration latency independently of infrastructure specification, highlighting the role of tool-internal execution pipelines in deployment performance.

The study provides a clear, reproducible methodology for assessing Infrastructure as Code (IaC) solutions in controlled environments, along with tool-specific insights. The experimental design illustrates that recurrent deployment testing can provide significant insights into real-world infrastructure performance when architectural and environmental variables are meticulously standardized.

Subsequent studies ought to broaden this framework to other avenues. Multi-cloud replication would help determine whether the observed performance disparities persist across provider ecosystems. Assessing containerized, serverless, and extensive microservices workloads would enhance architectural representativeness. Longitudinal experimentation could assess the consistency of tool performance over time and across service changes. Controlled usability studies can yield empirical insights regarding developer ergonomics and cognitive strain. Furthermore, incorporating long-term operational modeling—encompassing infrastructure drift management, performance reconciliation, and lifespan cost analysis—would enhance understanding of the sustained impact on output. The findings should be interpreted in light of several limitations, including the single-cloud-provider environment, the specific architectural pattern evaluated, and the focus on operational deployment characteristics rather than on full lifecycle infrastructure management.

Establishing a common benchmarking approach for Infrastructure as Code technologies would greatly improve methodological consistency across future investigations. Establishing these criteria would facilitate a more systematic, reproducible, and comparative assessment of infrastructure automation solutions in dynamic cloud environments.

Author Contributions: Conceptualization: D.R. and I.V.; data curation: I.V.; formal analysis: I.V. and M.B.; investigation: D.R. and I.V.; methodology: D.R. and I.V.; resources: D.R. and M.B.; supervision: D.R. and M.B.; validation: D.R. and I.V.; visualization: D.R. and M.B.; writing—original draft: D.R., I.V. and M.B.; writing—review & editing: D.R. and M.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

Table A1. Statistical analysis of the infrastructure destruction phase.

Tool	Mean (s)	Median	SD	Variance	Min	Max	Range	Skewness	Kurtosis
Terraform	466.23	464.0	23.98	575.08	427	527	100	0.60	0.35
Pulumi	513.73	511.0	30.68	941.03	428	569	141	−0.50	0.75
CloudFormation	578.40	573.0	49.52	2452.04	488	720	232	0.76	1.15

References

1. Rahman, A.; Williams, L. Characterizing Defective Configuration Scripts Used for Continuous Deployment. In *Proceedings of the 2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*; IEEE: New York, NY, USA, 2018; pp. 34–45. [\[CrossRef\]](#)
2. Dey, T.; Mockus, A. Effect of Technical and Social Factors on Pull Request Quality for the NPM Ecosystem. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*; IEEE: New York, NY, USA, 2020; pp. 1–11. [\[CrossRef\]](#)
3. Dalla Palma, S.; Di Nucci, D.; Palomba, F.; Tamburri, D.A. Within-Project Defect Prediction of Infrastructure as Code Using Product and Process Metrics. *IEEE Trans. Softw. Eng.* **2022**, *48*, 2086–2104. [\[CrossRef\]](#)
4. Chiari, M.; De Pascalis, M.; Pradella, M. Static Analysis of Infrastructure as Code: A Survey. In *Proceedings of the 2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*; IEEE: New York, NY, USA, 2022; pp. 218–225. [\[CrossRef\]](#)

5. War, A.; Nikiema, S.L.; Samhi, J.; Klein, J.; Bissyande, T.F. Security smells in infrastructure as code: A taxonomy update beyond the seven sins. *arXiv* **2025**, arXiv:2509.18761. [[CrossRef](#)]
6. Sokolowski, D.; Spielmann, D.; Salvaneschi, G. Automated Infrastructure as Code Program Testing. *IEEE Trans. Softw. Eng.* **2024**, *50*, 1585–1599. [[CrossRef](#)]
7. Sokolowski, D.; Spielmann, D.; Salvaneschi, G. The PIPr Dataset of Public Infrastructure as Code Programs. In *Proceedings of the 21st International Conference on Mining Software Repositories*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 498–503. [[CrossRef](#)]
8. Pahl, C.; Gunduz, N.; Sezen, Ö.; Ghamgosar, A.; El Ioini, N. Infrastructure as Code: Technology Review and Research Challenges. In *Proceedings of the 15th International Conference on Cloud Computing and Services Science*; SciTePress: Setúbal, Portugal, 2025; pp. 151–158. [[CrossRef](#)]
9. Ganesan, P. DevOps Automation for Cloud Native Distributed Applications. *J. Sci. Eng. Res.* **2020**, *7*, 342–347. [[CrossRef](#)]
10. Konala, P.R.R.; Kumar, V.; Bainbridge, D.; Haseeb, J. A Framework for Measuring the Quality of Infrastructure as Code Scripts. *arXiv* **2025**, arXiv:2502.03127. [[CrossRef](#)]
11. Dasari, H. Infrastructure as Code (IaC) Best Practices for Multi-Cloud Deployments in Enterprises. *Int. J. Netw. Secur.* **2025**, *5*, 174–186. [[CrossRef](#)]
12. Vaggu, H. Automating Infrastructure as Code (IaC): A Comparative Study of Terraform, Pulumi, and Kubernetes Operators. *Int. J. AI Big Data Comput. Manag. Stud.* **2025**, *6*, 1–9. [[CrossRef](#)]
13. Koneru, N.M.K. Infrastructure as Code (IaC) for Enterprise Applications: A Comparative Study of Terraform and CloudFormation. *Am. J. Technol.* **2025**, *4*, 1–29. [[CrossRef](#)]
14. Davidson, S.; Sun, L.; Bhasker, B.; Callot, L.; Deoras, A. Multi-IaC-Eval: Benchmarking Cloud Infrastructure as Code Across Multiple Formats. *arXiv* **2025**, arXiv:2509.05303. [[CrossRef](#)]
15. Yang, Z.; Guan, H.; Nicolet, V.; Paulsen, B.; Dodds, J.; Kroening, D.; Chen, A. Automated Cloud Infrastructure as Code Reconciliation with AI Agents. *arXiv* **2025**, arXiv:2510.20211. [[CrossRef](#)]
16. Begoug, M.; Ouni, A.; Chouchen, M. How Do Infrastructure-as-Code Practitioners Update Their Dependencies? An Empirical Study on Terraform Module Updates. In *Proceedings of the 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*; IEEE: New York, NY, USA, 2025; pp. 642–653. [[CrossRef](#)]
17. Tran, A.-D.; Sion, L.; Yskout, K.; Joosen, W. TerrARA: Automated Security Threat Modeling for Infrastructure as Code. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 269–280. [[CrossRef](#)]
18. Saavedra, N.; Ferreira, J.F.; Mendes, A. InfraFix: Technology-Agnostic Repair of Infrastructure as Code. *arXiv* **2025**, arXiv:2503.17220. [[CrossRef](#)]
19. Dandolu, S. Srikanth Dandolu Infrastructure as Code for Cloud-Native Data Platforms: Automation and Best Practices. *J. Comput. Sci. Technol. Stud.* **2025**, *7*, 451–488. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.